外文文献翻译

第一章 多尺度问题与数值均匀化

许多微观异质介质中的物理过程，如现代复合材料和功能材料，都可以用具有粗糙系数的偏微分方程（PDE）或具有复杂微观结构的区域来建模。鉴于这些过程的复杂性，其可靠的计算机模拟通常涉及构建适当的宏观（均匀化或有效）模型。我们将这类计算技术称为数值均匀化方法，它是更一般的多尺度方法的一个实例。本引言章节介绍了一些应用，简要回顾了多尺度方法的历史，并展示了本书的主要主题——局部正交分解方法如何与该领域内的其他方法相关联。

1.1 多尺度问题

在许多不可分离尺度上的异质微观结构，以及组成成分物理性质的高对比度，是现代复合材料和多功能材料优越性能的关键特征。然而，这些特征在计算机模拟中造成了重大困难。解析所有特征长度尺度的计算代价过高，而天真地忽略相关微观信息则会导致结果存疑，即使是在宏观感兴趣的尺度上也是如此。数值均匀化方法试图解决这一困境。它们以分层、并发和自适应的方式考虑相关微观信息，使得多尺度问题的可靠模拟最终在最先进的计算环境中变得可行。本书涵盖了过去十年发展起来的数值均匀化技术的设计、实现和数学分析——局部正交分解（LOD）方法。

LOD方法的可能应用之一是复合材料和多功能材料的力学分析。通过操纵其组成成分的特性和相对体积，可以使工程多相材料具有某些目标物理性质组合（例如，轻质、刚度、强电磁有序、能量转换）。为下一代性能定制结构开发新型多功能材料需要对微观结构进行拓扑优化，因此需要理解某些材料性质（电导率、渗透率等）如何依赖于可控变量（组成成分的热导率、相对体积、粒子形状、涂层和尺寸）。有时周期性微观结构是产生令人惊讶的宏观效应的关键，如负光学折射[52, 138]或其声学和地震类似物[59, 46, 32]。在许多其他情况下，正是与几何结构（如周期性和尺度分离）的偏离才在宏观尺度上引起理想的物理效应。一个代表性的例子是安德森局域化这一普遍波动现象，它存在于声波和光波中[87, 147]。对于玻色-爱因斯坦凝聚体背景下的物质波，在高振幅高振荡势阱下观察到安德森局域化，这些势阱表现出足够大的无序度，如[141]实验装置中使用的随机激光散斑势。多孔介质中的输运过程，如地下水流动[140, 148]或二氧化碳封存[122]，也面临许多相同的挑战，因为由于不同类型的土壤、岩石中的微观夹杂物或多孔地下岩层，渗透率具有快速变化的非周期性特征。纸张弹性特性的模拟是另一个超越周期性和尺度分离的多尺度应用例子[93]。纸张的宏观性质取决于图1.1所示的微观纤维配置。

在所有这些应用中，任何有意义的相关物理效应数值模拟都必须考虑整个计算域中的高度异质细尺度结构。如果宏观效应是感兴趣的，当使用标准有限元或有限体积方法时，计算负载很容易超出计算机能力。

1.2 数值均匀化方法简史

多尺度问题分析和数值技术发展的共同起点是稳态异质扩散方程：

$$-\mathrm{div}(A\nabla u) = f \qquad (1.1)$$

其中 A 是正定的快速变化扩散矩阵，f 是给定的源项。扩散矩阵建模了例如复合材料中的热或电导率、地球地下的渗透率，或在向量值设置中，复合材料的弹性特性。即使对于这个看似简单的模型问题，直接数值模拟（例如通过有限元方法或有限差分方法）也是具有挑战性的，因为扩散矩阵的空间变化需要被计算网格充分解析才能获得精确解。这个问题，我们将在第2章中更详细地研究，导致了许多数值算法的发展，通常被称为多尺度方法，始于20世纪80年代和90年代。

周期性和尺度分离的特殊情况（$$A(x) = A_\varepsilon(x) = A_1(x/\varepsilon)$$，其中 $\varepsilon > 0$ 是某个小周期，A_1 是1-周期系数）已使用均匀化理论进行了广泛研究。对于周期性数据，方程(1.1)的解，当 $\varepsilon \to 0$ 时，求解一个具有常数（有效）扩散矩阵 A_0 的类似方程。A_0 的构造涉及局部（在 ε 尺度上）椭圆问题的求解。通过求解局部问题来计算快速变化数据（或整个微分算子）的有效表示的想法是基本的，也是所有数值多尺度方法的基础。均匀化理论直接启发了多尺度有限元方法（MsFEM）[86, 51]和异质多尺度方法（HMM）[47]的发展，并用于分析这些方法的精度。这一发展始于20世纪90年代，至今仍是一个非常活跃的研究领域[2, 50, 38]。

另一个框架，也是在20世纪90年代发展起来的，是变分多尺度方法[88]，它使用弱形式的试验和检验空间的分解，采用（粗尺度）有限元插值。整个空间被分解为（粗尺度）有限元空间和（细尺度）余项空间，后者定义为插值的核。细尺度效应通过在余项空间中求解扩散问题被纳入粗尺度方程。最初这些细尺度问题通过分析技术近似，但后来它们在顶点片上被数值求解[109, 100, 90, 121]。为了使这种技术在数值上有用，余项空间中求解的细尺度问题的解需要对局部化右端数据呈指数衰减。这一点的数值证据在[109]中已经可以看到，但当时没有给出理论证明，保证精度的唯一方法是通过后验误差控制。

关于是否存在稳定且精确的方法超越解析均匀化关于尺度分离甚至周期性的强结构假设的问题长期悬而未决。[13]和[69]中证明了低正则性解空间存在通过某个任意粗糙的广义有限元空间（代表均匀化问题）的最优近似。然而，其中的构造包括计算代价过高的全局细尺度问题的完整求解或更复杂的特征

值问题的求解。一个高效且可行的构造，仅基于局部化微观胞元问题的求解，首次在[114]中于2014年给出并严格证明，后来在[80, 79, 81]中优化和推广，并在[96]中重新解释。该方法建立在变分多尺度方法及其在早期工作[109, 100, 90, 121]中引入的粗细尺度正交分解之上。这种多尺度方法，我们现在称之为LOD方法，为早期贡献提供了理论证明。

LOD的推导、分析和应用是本书的主题。虽然这是一个相当新近的发展，但该方法在过去五年中已经启发了众多新方法，例如粗糙多调和样条[128]、迭代数值均匀化[98]和Gamblets[124, 126]。连同许多其他同时期的发展，如多尺度混合-杂交方法[11]、AL基[69]和广义（多尺度）有限元方法[13, 50]及其各种衍生方法，这些方法形成了新一代数值均匀化方法，成功摆脱了均匀化理论的强结构假设，将可计算处理的确定性多尺度问题范围远远推向超越周期性和尺度分离。

第七章 实现方面

本章介绍LOD方法在MATLAB中的一些基本实现[116]。源代码可作为本书的补充材料获得。该实现针对在移动计算机上说明该方法的目的进行了优化。我们将简要讨论更适合高性能科学计算应用的替代实现。

7.1 一般说明

LOD方法可以应用于多种场景，其计算机实现可能相应变化。均匀化问题（如迄今研究的椭圆问题）的典型感兴趣区域是扩散系数在如此小的尺度上振荡，以至于无法被全局细网格和相应的有限元方法解析。对于这类应用，LOD可以通过将问题分解为可以独立处理的片上的较小问题来帮助使问题变得可处理，这在并行计算环境中可以加速，或者在其他情况下提高内存效率。注意在这种情况下，修正子不能存储，Petrov-Galerkin版本明显更优，因为其组装不需要不同单元上单元修正子的交互。这意味着LOD矩阵可以像往常一样在对粗单元的单次循环中组装。这在开源软件库GRIDLOD中得到了很好的实现[71]。

LOD方法计算机实现的非常详细的讨论在[56]中给出，相应的软件包在软件库DUNE中可用[17]。

在这本入门书中，我们不会重复[56]的讨论，而是提供一个更容易进入该方法的途径。为了将代码压缩到绝对最小，我们将假设可以访问全局细网格 $\mathcal{T}_h$ 和相应的有限元矩阵。在这种情况下，LOD方法可以看作一种压缩方案，它将具有 $O(\#\mathcal{T}_h)$ 个非零元素的大规模稀疏有限元矩阵压缩为与粗网格 $\mathcal{T}_H$ 相关的粗尺度稀疏矩阵，其中只需存储 $O(\#\mathcal{T}_H(\log \#\mathcal{T}_H)^d)$ 个非零元素，同时在很大程度上保持相应近似的精度。如果只对求解一个椭圆问题感兴趣，这个过程不会有回报。然而，在第8章将讨论的线

性和非线性特征值问题的背景下，情况就不同了。在那里，需要用同一矩阵求解一系列线性问题以求解特征值问题，因此如果需要超过几次迭代，维数降低将显著加速计算，如[56]所示。对于第9章将讨论的演化问题，预期会有更多的计算收益。

7.2 基本实现

本节说明使用商业软件MATLAB实现第5.1定义中引入的LOD方法标准版本[116]。它也应该可以在开源替代软件OCTAVE中执行[49]。源代码请参阅本书的补充材料，位于 bookstore.siam.org/sl05/bonus。它包括辅助函数 `refineMesh`、`assembleStiffnessMatrixDG` 和 `computeArea`，它们实现有限元代码中的标准任务。我们将在它们首次出现时简要解释它们。使用的其余函数是MATLAB的内置函数，有关详细信息，请参阅其文档。

模型问题

我们将考虑原型椭圆模型问题

$$-\mathrm{div}\, A\nabla u = f$$

在单位正方形 $\Omega = (0,1)^2$ 上，在 $\partial\Omega$ 上有齐次狄利克雷边界条件，这以前用于数值说明第3-5章的结果。扩散系数 A 是图3.1中所描绘的。右端 f 对 Ω 中的每个点 x 返回1。这个特定模型问题的数据由以下命令生成。

代码块

```
% 物理维度
d = 2;
% 设置随机数生成器种子（如果可能）
if exist('rng') rng(27); end
% 扩散系数A的界
alpha = 0.01;
beta = 1;
% 每个坐标方向的单元数
Neps = 2^6;
% 笛卡尔网格上的扩散系数
Aeps = alpha+(beta-alpha).*(randi(2,Neps,Neps)-1);
```

```
% 将Aeps转换为依赖于Omega中点（列表）的函数
A = @(x) Aeps((ceil(x(:,1)*Neps)-1)*Neps+(ceil(x(:,2)*Neps)));
% 右端
f = @(x) ones(size(x,1),1);
```

通过更改上述代码行可以轻松调整数据。

有限元网格

我们将使用初始三角剖分 $\mathcal{T}_0$ 的一致加密。三角网格通常由其顶点坐标列表（矩阵 p 的行）表示。三角形由其3个顶点在 p 中的（行）索引表示，按行存储在矩阵 t 中。为方便起见，两个字段 p 和 t 被组合成具有字段名 t 和 p 的结构数组。添加第三个字段 `Dnodes` 以存储边界顶点的索引。以下行定义了单位正方形的初始三角剖分。

代码块

```
% 顶点坐标列表
p = [0 0; 1 0; 1 1; 0 1];
% 通过顶点编号引用的三角形
t = [1 2 4; 2 3 4];
% 作为结构数组的Omega初始三角剖分
T0 = struct('t',t,'p',p,'Dnodes',[1;2;3;4]);
```

粗网格 $\mathcal{T}_H$ 和细网格 $\mathcal{T}_h$ 是初始网格的一致加密。它们由应用于 $\mathcal{T}_0$ 的一致加密次数 `Hlevel` 和 `hlevel` 来表征。LOD的局部化参数 ℓ 记为 `ell`。

代码块

```
%% LOD参数
Hlevel = 4;
hlevel = 8;
ell = 2;
```

一致网格加密由辅助函数 `refineMesh` 实现，它需要 $\mathcal{T}_0$ 格式的三角剖分（见第21行）并在第二个参数指定的加密步数后返回加密的三角剖分。

代码块

```
% 将初始网格加密Hlevel次
[TH] = refineMesh(T0,Hlevel);
```

拟插值

该方法的核心步骤是计算拟插值算子 $I_H = E_H \circ \Pi_H$ 的矩阵表示，它由到分片线性间断有限元的L^2-正交投影 Π_H 和示例3.1中引入的节点平均算子 E_H 组成。

代码块

```
% 连续到间断有限元函数的嵌入
cg2dgh = sparse(1:Nhdg,reshape(Th.t.',1,[]),1,Nhdg,Nh);
% 间断有限元的质量矩阵
Mhdg = kron(spdiags(computeArea(Th),0,NTh,NTh),[2 1 1;1 2 1;1 1 2]./12);
% 粗间断有限元质量矩阵的逆
B = kron(spdiags(1./computeArea(TH),0,NTH,NTH),[9 -3 -3;-3 9 -3;-3 -3 9]);
% 计算到粗间断有限元的L2投影
PiHdg = B*((P1dg.'*Mhdg)*cg2dgh);
```

对于节点平均E_H，我们考虑 `cg2dgh` 的粗尺度版本 `cg2dgH`。它的伪逆正是所需的 E_H 的矩阵表示 `EH`。

代码块

```
% 连续到间断有限元函数的嵌入
cg2dgH = sparse(1:NHdg,reshape(TH.t.',1,[]),1,NHdg,NH);
EH = spdiags(1./sum(cg2dgH,1).',0,NH,NH)*cg2dgH.';
```

综合起来，I_H 的矩阵表示 `IH` 可以计算。

代码块

```
% IH是EH和PiHdg的乘积
IH = EH*PiHdg;
```

单元片

在我们转向修正子问题之前，我们需要根据局部化参数 ℓ 识别单元片。设 `Ivt` 表示粗三角形和顶点的关联矩阵，即 `Ivt(i,j) ≠ 0` （等于1）当且仅当第 j 个单元包含第 i 个顶点。然后 `Ivt.'*Ivt` 的稀疏模式（由内置函数 `spones` 实现）提供粗三角形的关联矩阵，即 `Itt(i,j) ≠ 0` （等于1）当且仅当第 i 和第 j 个单元是相邻的（即它们共享至少一个顶点）。

代码块

```
% 粗单元和顶点的关联矩阵
Ivt = sparse(TH.t,repmat((1:NTH).',1,d+1),1,NH,NTH);
% 粗单元的关联矩阵
Itt = spones(Ivt.'*Ivt);
```

单元片将编码在矩阵 `patch` 中，其性质是 `patch(i,j) ≠ 0` 当且仅当第 i 个单元包含在关于第 j 个单元的片 $N^{\ell}(T_j)$ 中。

代码块

```
% 初始化片矩阵为单位矩阵
patch = speye(NTH);
% 连续应用Itt ell次
for k = 1:ell
    patch = Itt*patch;
end
% 将非零元素设为1，对角线元素设为2
patch = spones(patch)+speye(NTH);
```

数值修正子

对所有 $T \in \mathcal{T}_H$ 计算单元修正子 C_T^ℓ（如(4.10)中定义的），或更精确地说，它们的细尺度近似 $C_{T,h}^\ell$，是实现的核心。我们将单元修正子存储在元胞数组 `CT` 中。

代码块

```
CT = cell(1,NTH);
% 遍历粗单元
for k=1:NTH
```

在循环内，我们处理一个固定的单元 T（第 k 个单元）并计算相应的修正子 $C_{T,h}^\ell$。由于只需计算其在与 T 的顶点相关联的节点基函数 $\Lambda_{T,1}$、$\Lambda_{T,2}$、$\Lambda_{T,3}$ 上的作用，我们只需计算 $\phi_i \in W_h^\ell(T)$，$i = 1, 2, 3$，使得

$$\int_{N^\ell(T)} (A\nabla\phi_i)\cdot\nabla w_h\,dx = \int_T (A\nabla\Lambda_{T,i})\cdot\nabla w_h\,dx \quad \text{对所有 } w_h \in W_h^\ell(T)$$

构建和求解粗LOD系统

给定LOD基 `G`，粗系统容易组装。自由度是粗网格的内部顶点。LOD刚度矩阵是通过从两侧乘以 `G` 和 `cg2dgh` 并限制到粗自由度来压缩全局细尺度间断刚度矩阵 `Shdg` 到粗尺度的结果。LOD基函数对右端函数 f（在粗顶点处插值）进行积分。结果线性系统可以使用内置稀疏直接求解器求解。

代码块

```
% 粗自由度（内部顶点）
dofH = true(NH,1);
dofH(TH.Dnodes) = 0;
% LOD刚度矩阵
G0 = G(:,dofH);
SHLOD0 = ((G0.'*(cg2dgh.'*Shdg))*cg2dgh)*G0;
% 右端
rhs = G0.'*(cg2dgh.'*(Mhdg*(cg2dgh*(P1*f(TH.p)))));
```

```
% 求解粗线性系统
uH = zeros(NH,1);
uH(dofH) = SHLOD0\rhs;
```

向量 `uH` 包含基表示中LOD近似的系数。当解释为粗有限元函数时，它是LOD解的有限元部分 $u_{H,\ell,h}$，近似参考解 $u_h \in V_h$ 的拟插值 $I_H u_h$。

表示 u_H^{ms}（即 $u_{H,\ell,h}^{ms}$）的完整LOD近似通过将基矩阵 `G` 应用于 `uH` 来恢复。

代码块

```
% 重建细尺度信息以计算LOD近似
uHms = G*uH;
```

LOD近似相对于参考解 u_h 的误差在能量范数中计算为 2.47816e-02，在 $L^2(\Omega)$ 范数中计算为 1.66544e-03。

我们最后强调，代码容易修改，使全局细网格的计算可以避免。只需在修正子循环中包含（局部）网格加密和（局部）拟插值计算即可。